

Week 9 Practical

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
divya@Divyasree:~$ mkdir file_copy_project
divya@Divyasree:~$ cd file_copy_project
divya@Divyasree:~/file_copy_project$ nano low_level_copy.c
divya@Divyasree:~/file_copy_project$ nano standard_copy.c
divya@Divyasree:~/file_copy_project$ gcc -Wall -O2 low_level_copy.c -o low_level_copy
divya@Divyasree:~/file_copy_project$ gcc -Wall -O2 standard_copy.c -o standard_copy
divya@Divyasree:~/file_copy_project$ ls
low_level_copy  standard_copy
low_level_copy.c  standard_copy.c
divya@Divyasree:~/file_copy_project$ dd if=/dev/zero of=source.txt bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0470403 s, 2.2 GB/s
divya@Divyasree:~/file_copy_project$ ls -lh source.txt
-rw-r--r-- 1 divya divya 100M Sep 30 16:14 source.txt
divya@Divyasree:~/file_copy_project$ time ./low_level_copy source.txt low_level_output.txt
Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.069s
user    0m0.004s
sys     0m0.072s
divya@Divyasree:~/file_copy_project$ time ./standard_copy source.txt standard_output.txt
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

real    0m0.075s
user    0m0.004s
sys     0m0.078s
divya@Divyasree:~/file_copy_project$ ls -lh source.txt low_level_output.txt standard_output.txt
-rw-r--r-- 1 divya divya 100M Sep 30 16:14 low_level_output.txt
-rw-r--r-- 1 divya divya 100M Sep 30 16:14 source.txt
-rw-r--r-- 1 divya divya 100M Sep 30 16:14 standard_output.txt

-rw-r--r-- 1 divya divya 100M Sep 30 16:14 standard_output.txt
divya@Divyasree:~/file_copy_project$ cmp source.txt low_level_output.txt
divya@Divyasree:~/file_copy_project$ cmp source.txt standard_output.txt
divya@Divyasree:~/file_copy_project$ sha256sum source.txt
sha256sum low_level_output.txt
sha256sum standard_output.txt
20492a4d0d84f8beb1767f6616229f85d44c2827b64bdbfb260ee12fa1109e0e  source.txt
20492a4d0d84f8beb1767f6616229f85d44c2827b64bdbfb260ee12fa1109e0e  low_level_output.txt
20492a4d0d84f8beb1767f6616229f85d44c2827b64bdbfb260ee12fa1109e0e  standard_output.txt
divya@Divyasree:~/file_copy_project$ for i in {1..5}; do time ./low_level_copy source.txt low_${i}.txt; done
Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.066s
user    0m0.008s
sys     0m0.064s
Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.068s
user    0m0.000s
sys     0m0.074s
Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.074s
user    0m0.004s
sys     0m0.076s
Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.072s
user    0m0.000s
```

```

Current source file position: 104857600 bytes
File copied successfully using low-level I/O.

real    0m0.079s
user    0m0.005s
sys     0m0.082s
divya@Divyasree:~/file_copy_project$ for i in {1..5}; do time ./standard_copy source.txt standard_$i.txt;
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

real    0m0.069s
user    0m0.004s
sys     0m0.071s
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

real    0m0.076s
user    0m0.017s
sys     0m0.065s
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

real    0m0.111s
user    0m0.017s
sys     0m0.094s
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

real    0m0.116s
user    0m0.009s
sys     0m0.118s
Current source file position: 104857600 bytes
File copied successfully using standard library I/O.

```

Code :

```

GNU nano 7.2 low_level_copy.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <errno.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    // Open source file
    src = open(argv[1], O_RDONLY);

    if (src == -1) {
        perror("Error opening source file");
        return 1;
    }

    // Open/create destination file

```

```

dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (dest == -1) {
    perror("Error opening destination file");
    close(src);
    return 1;
}

// Read and write using system calls
while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0) {

    bytes_written = write(dest, buffer, bytes_read);

    if (bytes_written != bytes_read) {
        perror("Error writing file");
        close(src);
        close(dest);
        return 1;
    }
}

if (bytes_read == -1) {
    perror("Error reading file");
}

// Demonstrate lseek()
off_t position = lseek(src, 0, SEEK_CUR);

if (position != (off_t)-1) {

```

```

        (long)position);
    }

    close(src);
    close(dest);

    printf("File copied successfully using low-level I/O.\n");

    return 0;
}

```

```

GNU nano 7.2 standard_copy.c
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    FILE *src;
    FILE *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    // Open source file
    src = fopen(argv[1], "rb");

    if (src == NULL) {
        perror("Error opening source file");
        return 1;
    }

    // Open/create destination file
    dest = fopen(argv[2], "wb");

    if (dest == NULL) {

```

[Read 60 lines]

```

        fclose(src);
        return 1;
    }

    // Read and write using standard library functions
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0) {
        if (fwrite(buffer, 1, bytes_read, dest) != bytes_read) {
            perror("Error writing file");
            fclose(src);
            fclose(dest);
            return 1;
        }
    }

    // Display current file position
    long position = ftell(src);

    if (position != -1L) {
        printf("Current source file position: %ld bytes\n", position);
    }

    fclose(src);
    fclose(dest);

    printf("File copied successfully using standard library I/O.\n");

    return 0;
}

```

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```

divya@Divyasree:~$ mkdir io_redirection
divya@Divyasree:~$ cd io_redirection
divya@Divyasree:~/io_redirection$ nano io_redirect.c
divya@Divyasree:~/io_redirection$ echo "Hello Meghana!" > input.txt
divya@Divyasree:~/io_redirection$ echo "This text is coming from the input file." >> input.txt
divya@Divyasree:~/io_redirection$ cat input.txt
Hello Meghana!
This text is coming from the input file.
dup2() redirected standard input and output.
divya@Divyasree:~/io_redirection$ gcc -Wall -o io_redirect io_redirect.c
divya@Divyasree:~/io_redirection$ ls
input.txt  io_redirect  io_redirect.c
divya@Divyasree:~/io_redirection$ ./io_redirect
I/O redirection completed successfully.
Input was read from input.txt
Output was redirected to output.txt
divya@Divyasree:~/io_redirection$ cat output.txt
Hello Meghana!
This text is coming from the input file.
dup2() redirected standard input and output.
divya@Divyasree:~/io_redirection$ |

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int input_fd, output_fd;

    // Create input file
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0) {
        perror("Error opening input.txt");
        return 1;
    }

    // Create/open output file
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0) {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }
}

```

```

    if (output_fd < 0) {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        close(input_fd);
        close(output_fd);
        return 1;
    }

    if (pid == 0) {
        // Child process

        // Redirect standard input
        if (dup2(input_fd, STDIN_FILENO) == -1) {
            perror("dup2 stdin");
            exit(1);
        }

        // Redirect standard output
        if (dup2(output_fd, STDOUT_FILENO) == -1) {
            perror("dup2 stdout");
            exit(1);
        }
    }
}

```

```

        exit(1);
    }

    // Original descriptors are no longer needed
    close(input_fd);
    close(output_fd);

    // Execute command
    execlp("cat", "cat", NULL);

    // If execlp fails
    perror("execlp");
    exit(1);
}

// Parent process
close(input_fd);
close(output_fd);

wait(NULL);

printf("I/O redirection completed successfully.\n");
printf("Input was read from input.txt\n");
printf("Output was redirected to output.txt\n");

return 0;
}
|

```